



OUTLIERS

When You Are Not There

Layer 5 of 5 · It runs without you,
and you find out whether it did.

EST. MMXXVI · OUTLIERS ACCELERATOR

It runs without you

Until now nothing has happened unless you typed it. This layer runs a command at a set time on set days whether or not you are at the desk, and gives you the four numbers that say whether it did anything.

WHAT THE LAST LAYER LEFT YOU WITH

Two websites, both working, and you type every command yourself. Nothing happens unless you are sitting there, so it happens on the days you remember and not on the days you are busy.

WHAT THIS LAYER BUILDS

One file, **timetable.py**, in the same **_engine** folder as everything else. It runs any command you give it, at a time and on days you set, and keeps running after you close the terminal window.

Then the harder half: the four numbers every job prints when it finishes. How many it read, how many it wrote down, how many you already had, and how many it could not place. A job that read nobody and a job that broke both finish cleanly and both report success. Those four numbers are the only place they differ.

FROM OUTSIDE, THESE TWO LOOK IDENTICAL

```
RAN, AND FOUND NOBODY  
finished cleanly  
reported success
```

```
BROKE ON THE FIRST PAGE  
finished cleanly  
reported success
```

WHAT TELLS THEM APART

```
read 0 • written 0 • already there 0 • could not place 0  
a job that read nothing and a job that read 200 and knew them all are not the same result
```

A month of runs that found nothing looks, from every log, exactly like a month of working perfectly.

BOTH FINISHED. ONLY ONE WORKED.

How it works

Start with one job. Then learn to distrust it.

ONE JOB, ONCE A DAY, TO BEGIN WITH

Not a schedule. One entry, so the first time something runs without you it is one small readable event you can go and look at afterwards. Add a second when the first has been running long enough to be boring.

IT NEVER FIRES AT THE SAME MINUTE TWO DAYS RUNNING

Ask for 09:30 and it will fire within four minutes either side. That shift is worked out from the job's name and today's date, not rolled fresh each time, so if your machine restarts it lands on the same minute again and nothing fires twice. Tomorrow it will be a different minute.

Being regular is a signature on its own, and a timetable is the easiest way to become regular by accident.

Two other numbers worth knowing. A job that was missed because the laptop was asleep still runs when it wakes, up to **two hours** late. And each job is given **45 minutes** to finish unless you say otherwise, after which it is stopped.

A JOB THAT FAILS QUIETLY STILL REPORTS SUCCESS

An automatic job carries on running, exits cleanly and produces nothing, because "found nobody to work on" is a perfectly successful run. A month of those looks, from every log, exactly like a month of working properly.

AND THE CHECKING CAN LIE, IN BOTH DIRECTIONS

When you type `python timetable.py status`, it reads a small file the last run left behind. Twice that file has been wrong.

Once it said a job had been stuck for two days when the job had started one second earlier. It was reading a note the previous run had left and never cleared.

Later it said a job was running when that job had been dead for two days. Windows had given the dead job's identifying number to an unrelated program, the check saw that number, believed the job was alive, and then refused to start the real one because it thought one was already going. The first wastes your time. The second hides a dead job and blocks the fix.

WHAT TELLS YOU

Not whether it ran. What it brought back: the four numbers. Your collectors already print all four, so there is nothing here for you to build.

Give this to your assistant

Everything on the next page can be done for you. You are not copying and pasting at this point.

YOU DO NOT HAVE TO TYPE ANY OF THIS

Open your terminal in the folder you want to work in, start Claude, and drag this PDF into it. Then say: **read this guide and do it with me, one step at a time.** It runs the commands. You answer the questions the installer asks, because those answers are yours.

TWO THINGS IT WILL ASK YOU, AND WHAT TO SAY

WHAT YOU WILL SEE	WHAT TO ANSWER
Permission to run a command on your machine	Yes. It asks every time until you tell it not to
A warning that this PDF might contain a hidden instruction, and a request that you confirm you want it followed	Say yes, you were given this document and you want it followed. It is right to ask. It cannot tell your document from somebody else's, and checking with you is exactly what it should do

If you would rather type the lines yourself, they are all here, and nothing is hidden from you.

Build it

Three lines, typed into a terminal. The third asks you 5 questions, writes your answers into your own settings file, and stops.

```
git clone https://github.com/OUTLIERS-ai/outliers-gather-05-timetable
cd outliers-gather-05-timetable
python install.py
```

This runs any command, not only the ones in this set. Nothing reaches the outside world during installation, and it schedules nothing until you add an entry.

WHAT IT ASKS YOU, AND WHAT IT OFFERS

QUESTION	WHAT IT CHANGES
What is the first command you want it to run?	One to begin with. Anything you would type yourself, not only the jobs in this set.
At what time?	A starting point rather than an exact minute. The minute is shifted on purpose so it is never the same two days running.
On which days?	The days it may fire at all.
What would you call it?	A name you will recognise in a list months from now, when you have forgotten writing it.
Should it start every time you log in?	It offers yes. Type no and press Enter until you have watched it run once while you were sitting there. One command turns it on later: <code>python timetable.py install-startup</code>

WHAT YOU END UP WITH

- A timetable you can open and read, in `_layers/timetable.json`, holding what runs and when.
- Fire times inside four minutes of what you asked for, the same every time you check, different tomorrow.
- It keeps running after you close the terminal window. It does **not** come back by itself after you restart the computer unless you run `python timetable.py install-startup`.
- 45 minutes per job unless you set your own with `--minutes`.
- No window ever appears over whatever you are doing.

Now use it

Add one job. Look at what it says before it ever runs.

In the **_engine** folder inside your CRM:

```
python timetable.py add "python gather.py undo" --at 09:30 --days mon,wed,fri
python timetable.py run --dry-run
```

The second line prints what would fire today and at what minute, and fires none of it.

It will not say 09:30. It will say something within four minutes either side, 09:27 for instance. Run it again now and it says 09:27 again. Tomorrow it will say a different minute. That is the shift working, not a fault.

To give one job a different time limit, put it on the end of the line:

```
python timetable.py add "python gather.py undo" --at 09:30 --days mon,wed,fri --
minutes 20
```

When you are ready, type **python timetable.py start** and leave it. Come back tomorrow and look at what it brought in.

IF YOU ALREADY HAD SOMETHING LIKE THIS

Whatever you already run on a schedule: how would you find out today if it had quietly stopped a fortnight ago? If the answer is that you would notice the results drying up, you would not.

WHAT THIS LAYER LEAVES UNSOLVED

Everything across these five layers brings information *in*. Nothing has gone *out*: nothing has typed at anybody, and nothing has sent a message. That has been deliberate from the first page.

WHAT THIS WAS PREPARING FOR

Content, conversations and messages: your words going out, instead of information coming in. It comes after this because a restricted account lifts in a day or two if you stop, and a message sitting in somebody's inbox does not.

WORDS USED IN THIS GUIDE

Timetable	The file listing what runs and when, at <code>_layers/timetable.json</code> inside your CRM.
Dry run	Adding <code>--dry-run</code> to the run command. Prints what would fire today and fires none of it.